

OPPerTune

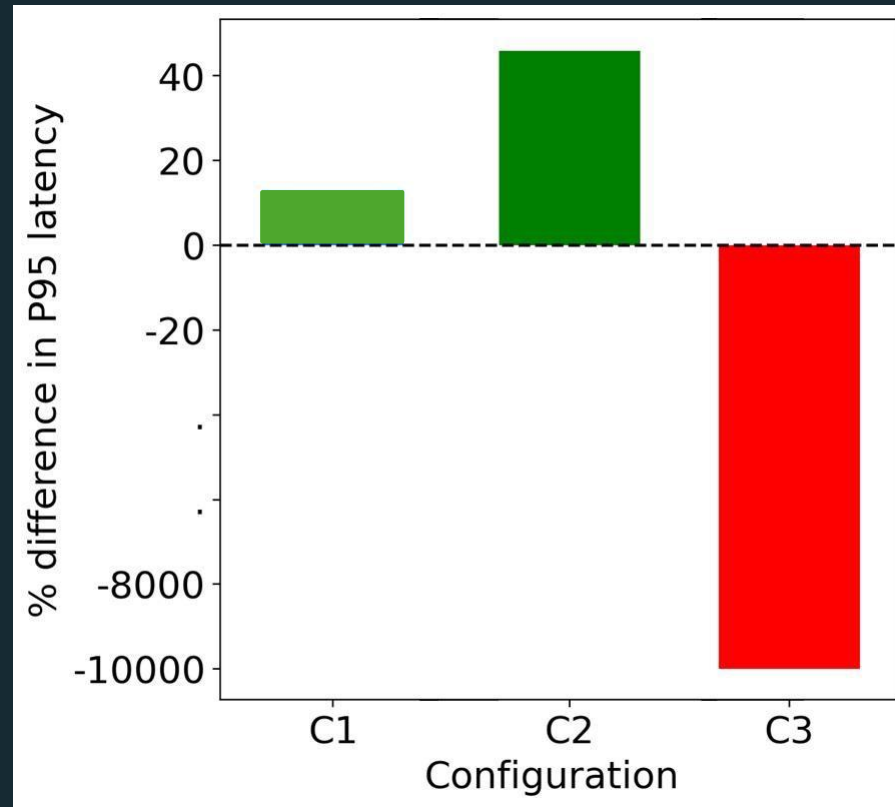
Post-Deployment Configuration Tuning of Services Made Easy

Gagan Somashekar*^{1,3}, **Karan Tandon***², Anush Kini², Chieh-Chun Chang⁴, Petr Husak⁴,
Ranjita Bhagwan², Mayukh Das³, Anshul Gandhi¹, Nagarajan Natarajan²

Presented By: Nitin Kedia

Motivation

- Application performance depends on its configuration



Microservices-based cloud app

Motivation

- Very large configuration space
 - N = Number of microservices
 - P = Parameters per microservice
 - C = Number of parameter values
 - **Total possible configurations** $\approx C^{N*P}$
- Why can't this space be tackled offline as in TUNA?
 - Workload changes online
 - Infrastructure changes online
 - Features are introduced frequently
 - Running TUNA after each of these offline is not feasible.
- Need to handle both Numerical and Categorical parameters?

Motivation

- Very large configuration space
 - N = Number of parameters
 - P = Parameter values
 - C = Number of parameter values
 - **Total possible configurations** $\propto N * P$
- Why can't this space be searched?
 - Workload changes
 - Infrastructure changes
 - Features are introduced frequently
 - Running TUNA after each of these offline is not feasible.

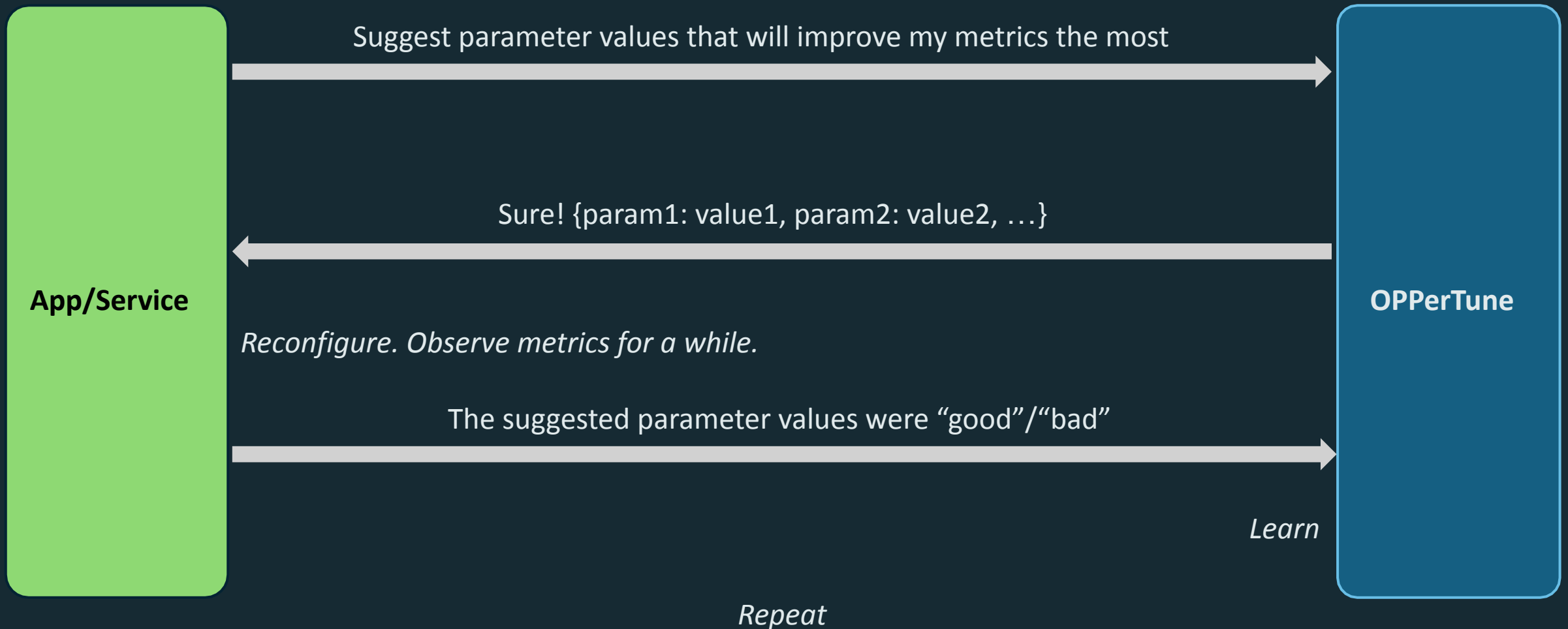
How to find the optimal configuration to maximize performance *while the application is deployed in production?*

How to find the optimal configuration in minimum steps?

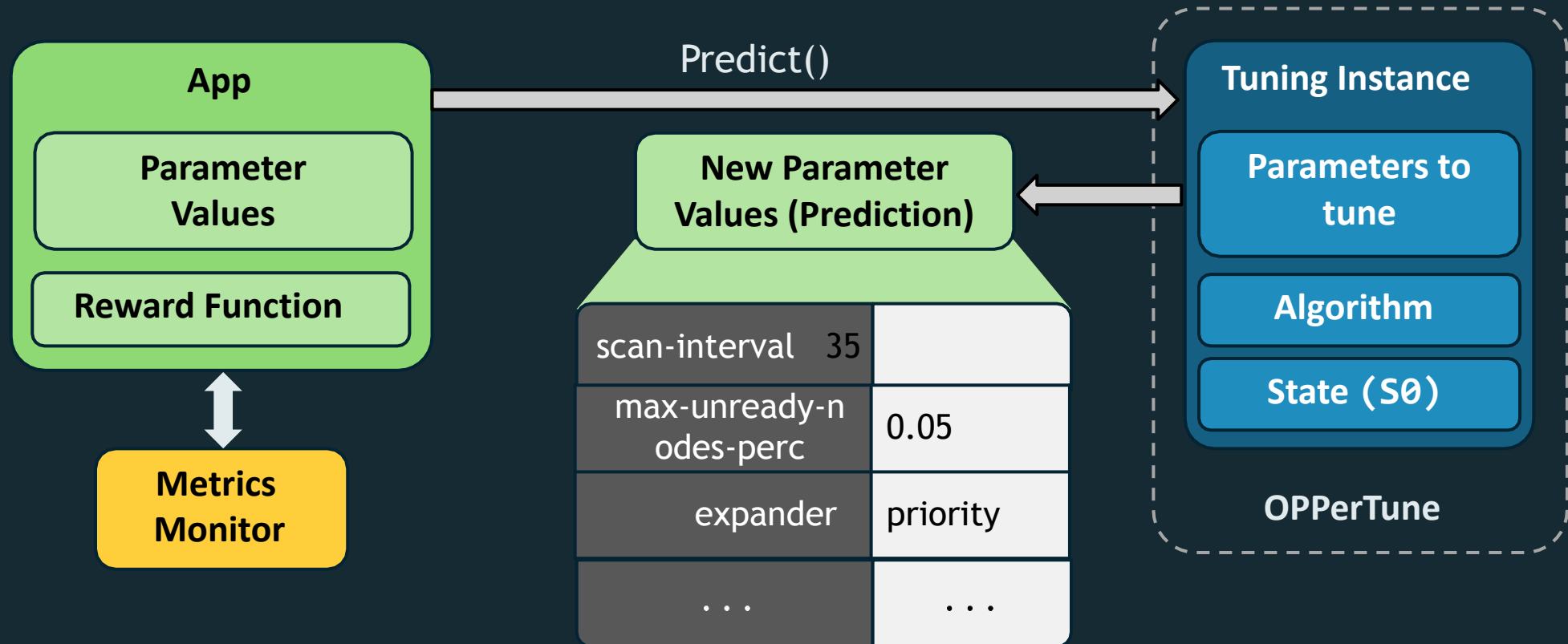
Need for a new framework

	CherryPick (NSDI '17)	μ Tune (OSDI '18)	OPTIMUS CLOUD (ATC '20)	KEA (SIGMOD'21)	SelfTune (NSDI '23)	OPPerTune
Handles numerical and categorical parameters	✓	✓	✓	✓	✗	✓
Can scope the problem	✗	✗	✗	✗	✗	✓
Filter parameters to tune	✗	✗	✓	✗	✗	✓
Low Sample Complexity	✓	✗	✗	✗	✓	✓
Application-independent	✓	✓	✓	✗	✓	✓
Supports online learning	✓	✗	✗	✓	✓	✓
Handles dynamic objective function	✗	✓	✓	✓	✓	✓
End-to-end framework*	✗	✗	✓	✓	✗	✓

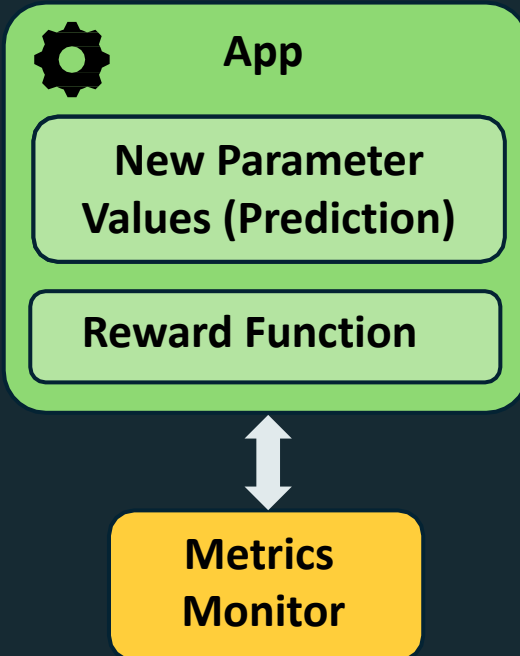
OPPerTune (Optimal Post-deployment Performance Tuner)



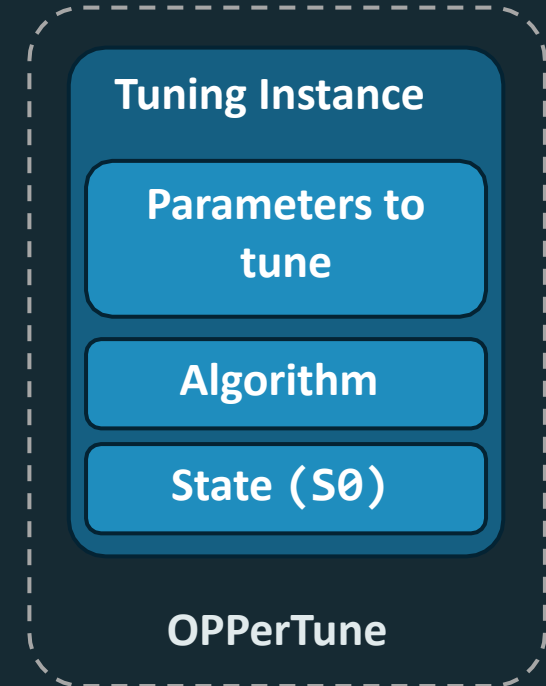
OPPerTune – Tuning Iteration



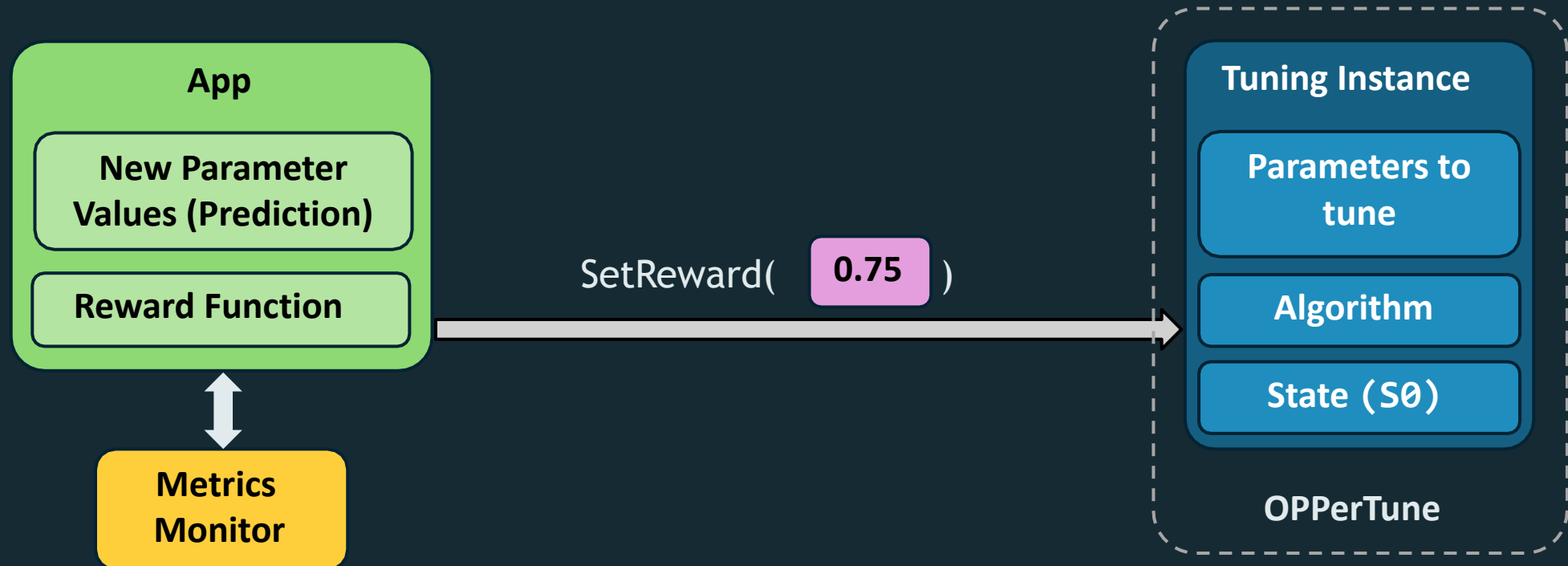
OPPerTune – Tuning Iteration



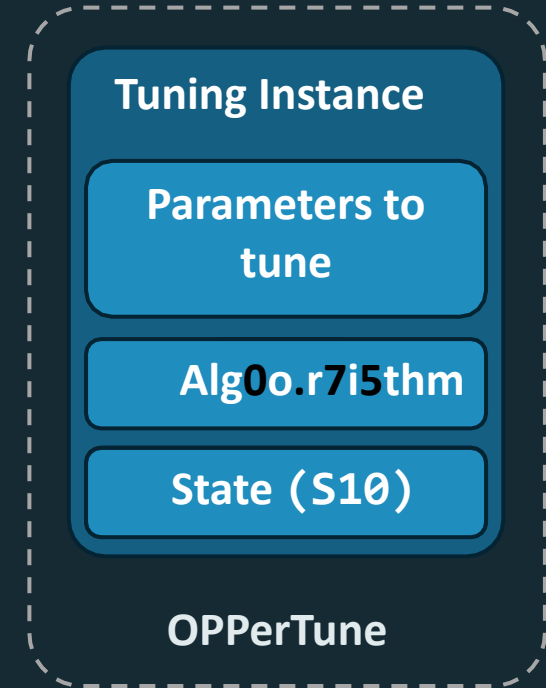
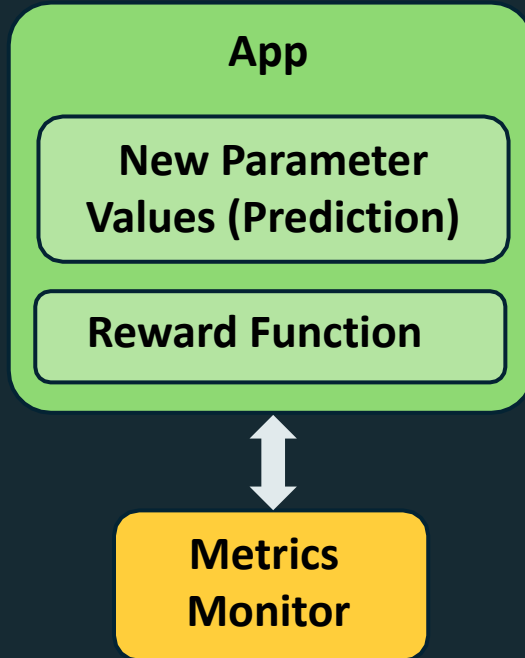

Observe metrics for a
few minutes/hours/days



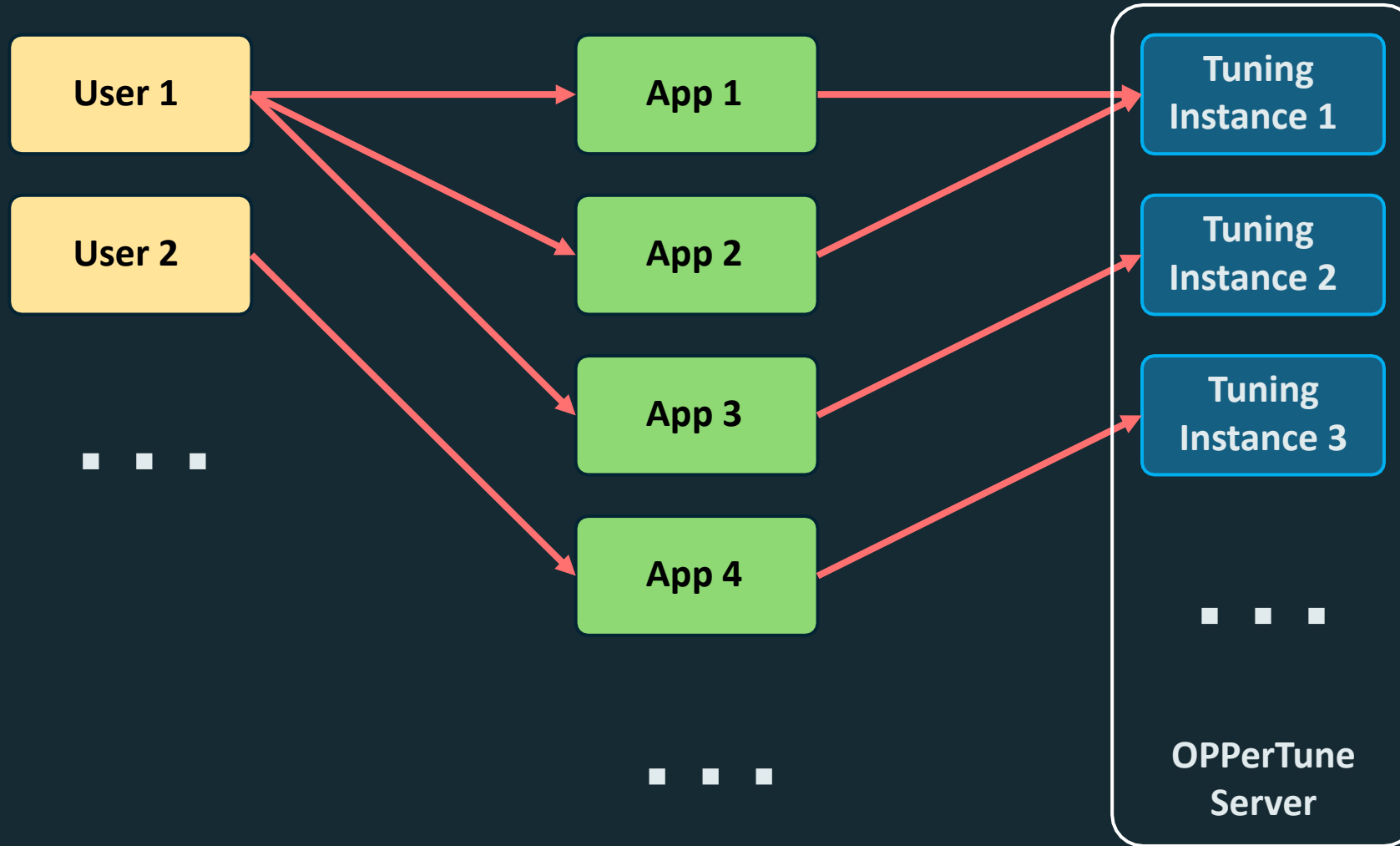
OPPerTune – Tuning Iteration



OPPerTune – Tuning Iteration



Tuning at scale



Challenge 1

Hybrid parameter space

Hybrid parameter space

- Jointly tuning numerical and categorical parameters for optimal performance

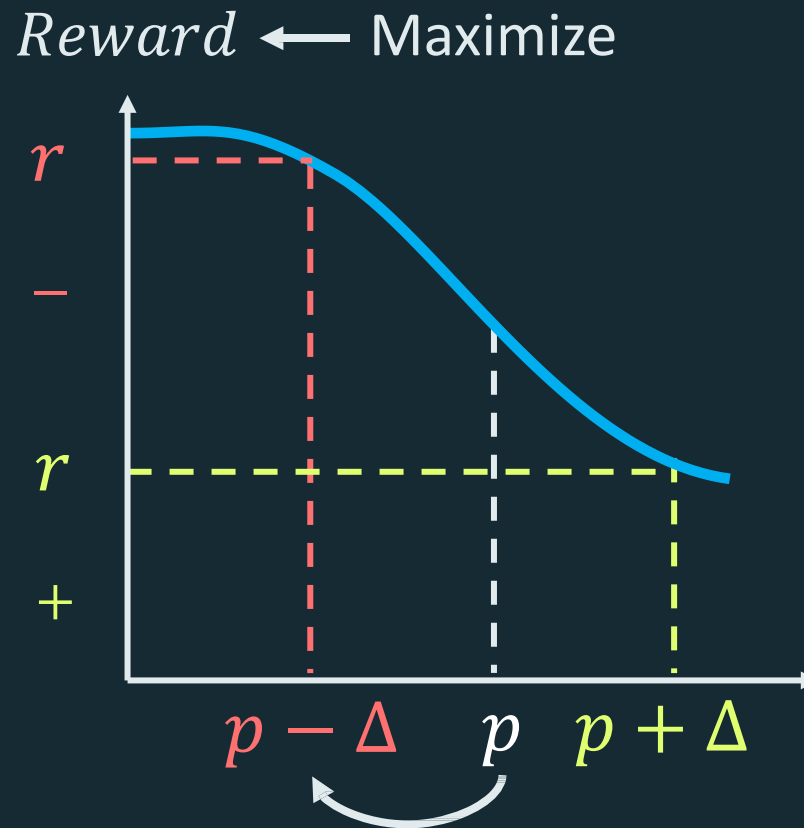
What about existing algorithms?

- Gradient-based algorithms: **Inapplicable due to lack of continuity**
- Traditional bandit algorithms & Deep RL techniques: **Sample-inefficient**
- Bayesian optimization: **Needs multiple samples for the same reward function**

Our solution - HybridBandits

- Scales linearly with number of numerical parameters and total categorical combinations
- Combines two algorithms – one for numerical and another for categorical parameters

HybridBandits (Numerical Parameters)

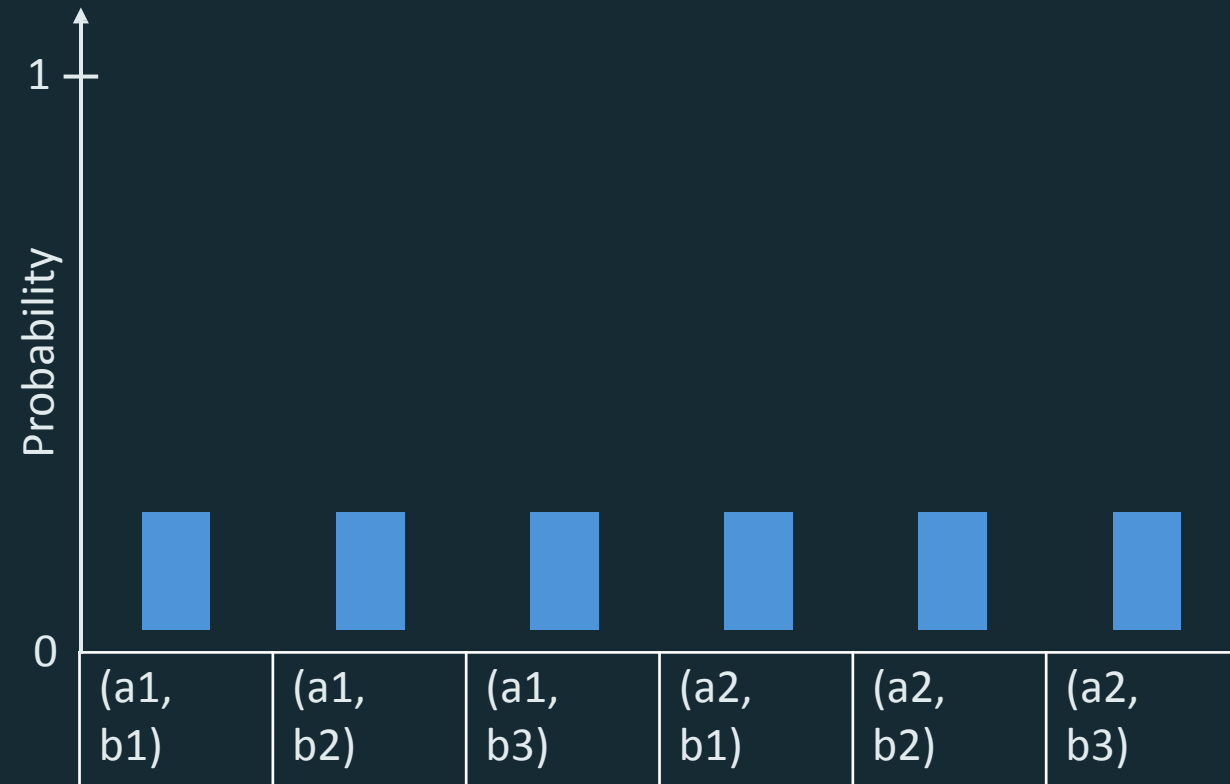


HybridBandits (Categorical Parameters)

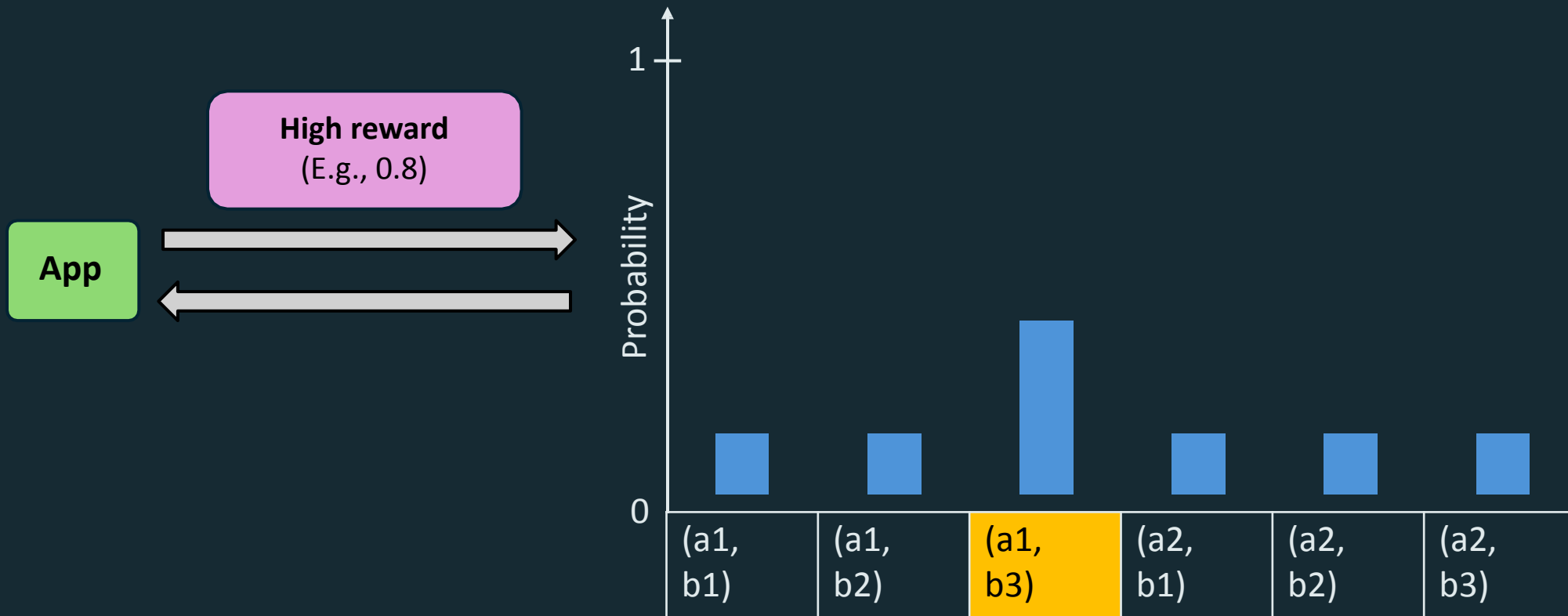
App

Name	a
Value	a2
Categories	0. a1 1. a2

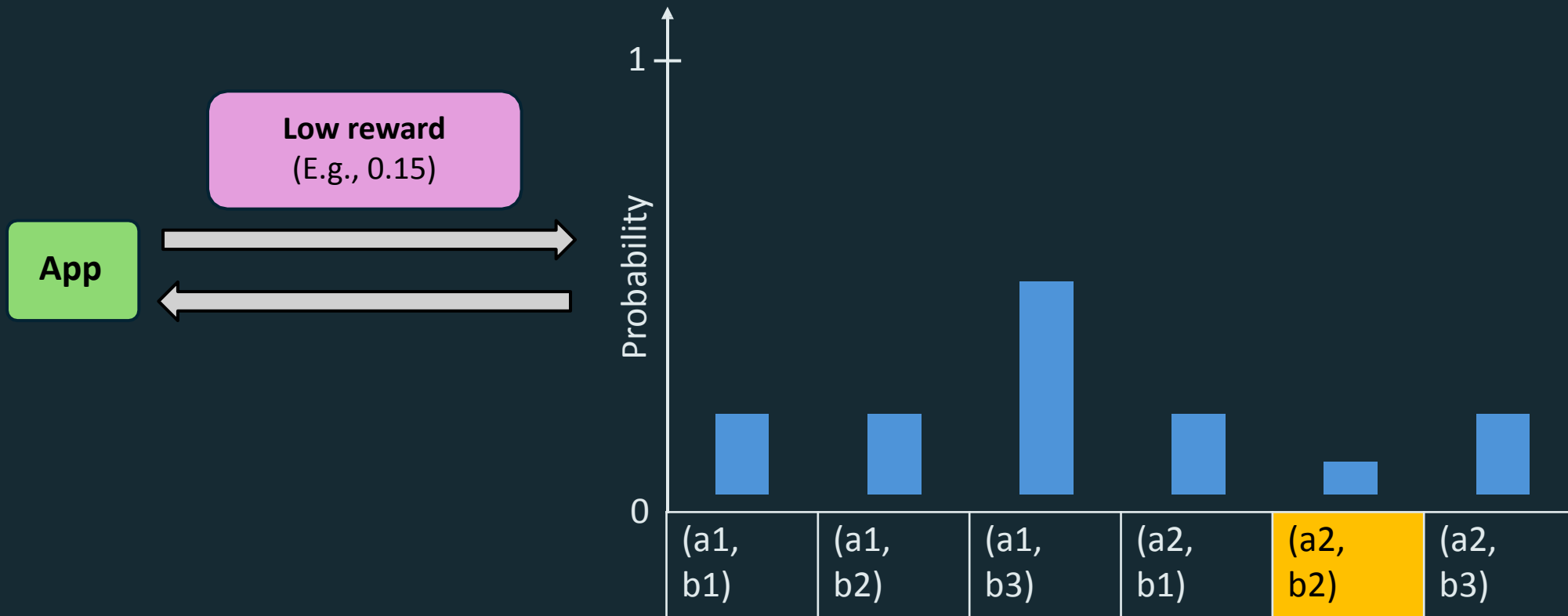
Name	b
Value	b1
Categories	0. b1 1. b2 2. b3



OPPerTune – HybridBandits (Categorical Parameters)



OPPerTune – HybridBandits (Categorical Parameters)



Our solution - HybridBandits

- Combines two algorithms – one for numerical and another for categorical parameters
- Scales linearly with number of numerical parameters and total categorical combinations

Challenges

- Two different algorithms tuning different types of parameters
- Need to work with a single reward value

Evaluation – DeathStarBench (Social Network app)



Death Star





Login

[Sign Up](#)

DeathStar

Search

Search

Post

Contacts

Zixiao



About

Edit

User Name: Zixiao

Gender: Male

Phone: +1(123)456-7890

Address:
100 Campus Rd.

 Zixiao

20:35 Fri May 15 2020

@jackxu

 Like  Comment  Share

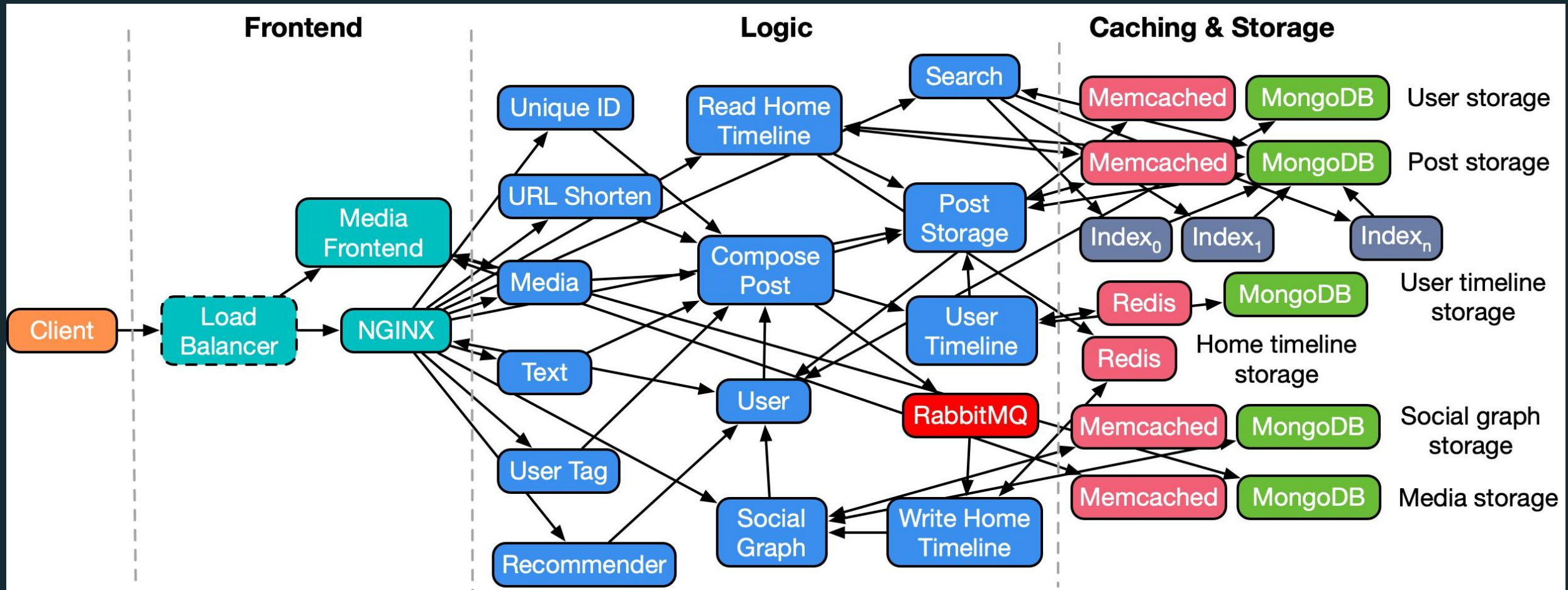
 Zixiao

20:35 Fri May 15 2020

Text Only Post

 Like  Comment  Share

Evaluation – DeathStarBench (Social Network app)

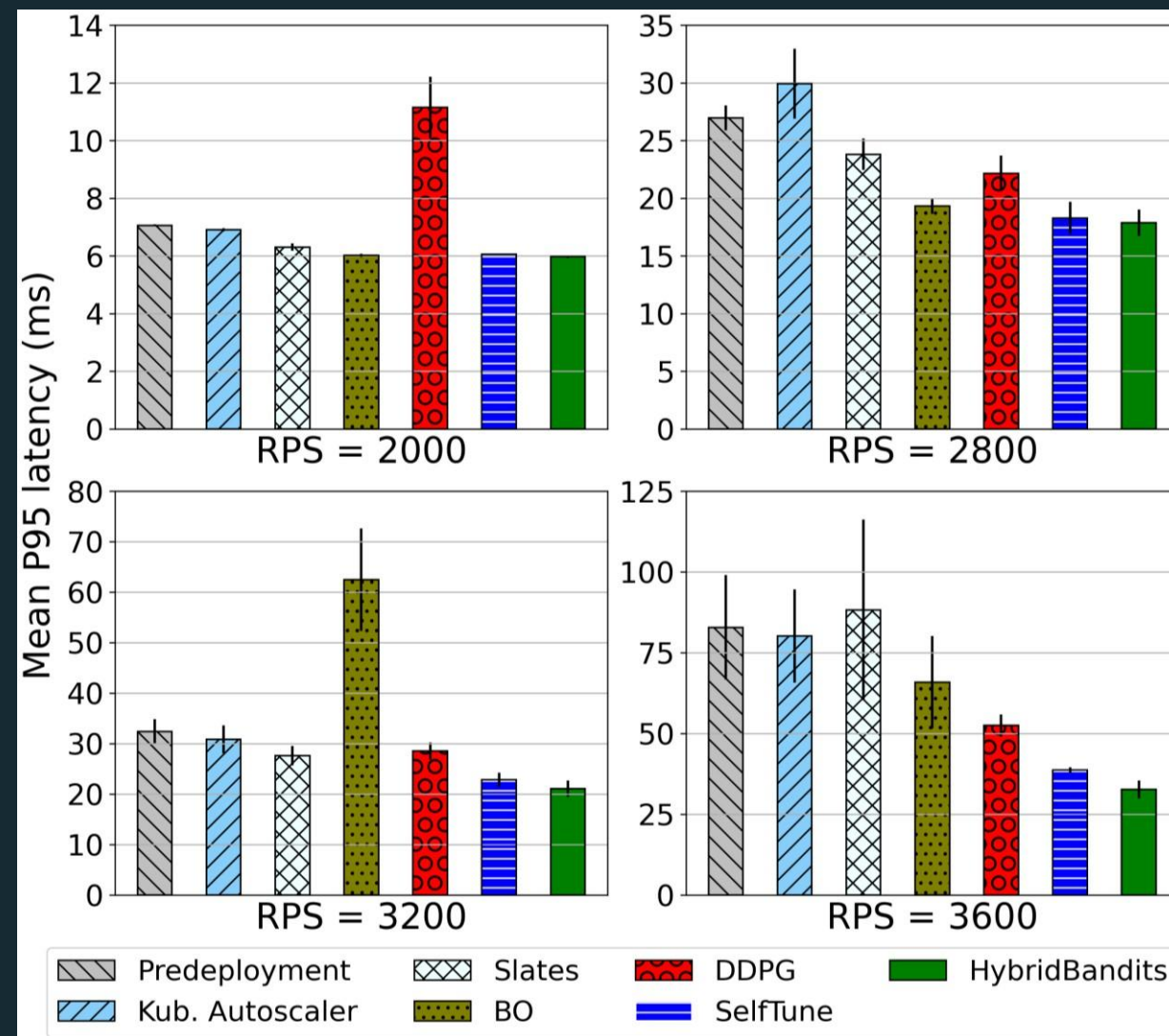


Effectiveness of HybridBandits

- **Workload:** Constant RPS on the DeathStarBench Social Network app
- Fixed budget of 50 iterations
- An iteration involves
 1. Configuring the app with the parameters returned by the algorithm
 2. Running workload for 20 minutes
 3. Measuring the P95 latency

Takeaway

About 15%-20% reduction in the tail latency of the application, relative to SelfTune, especially at higher workloads (when the shared resources in the VMs are strained).

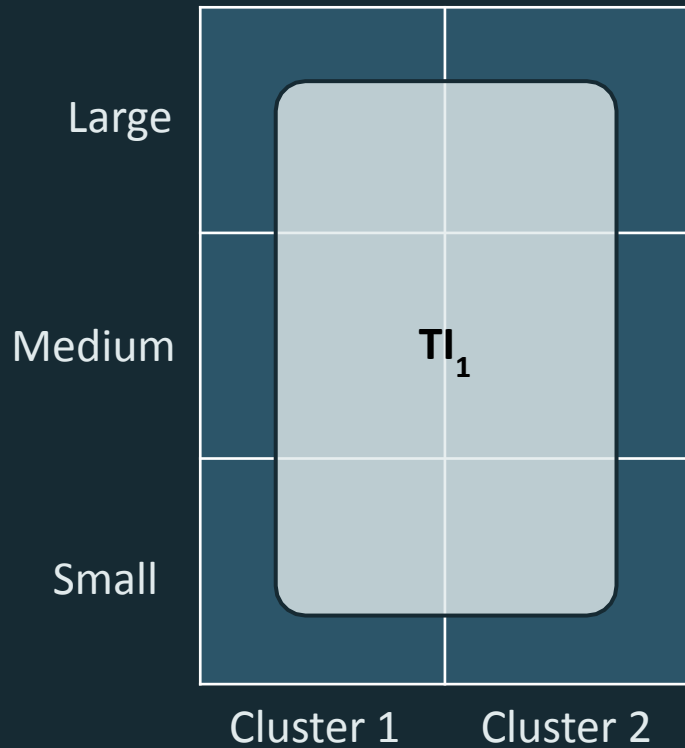


Challenge 2:

Scoping the tuning problem

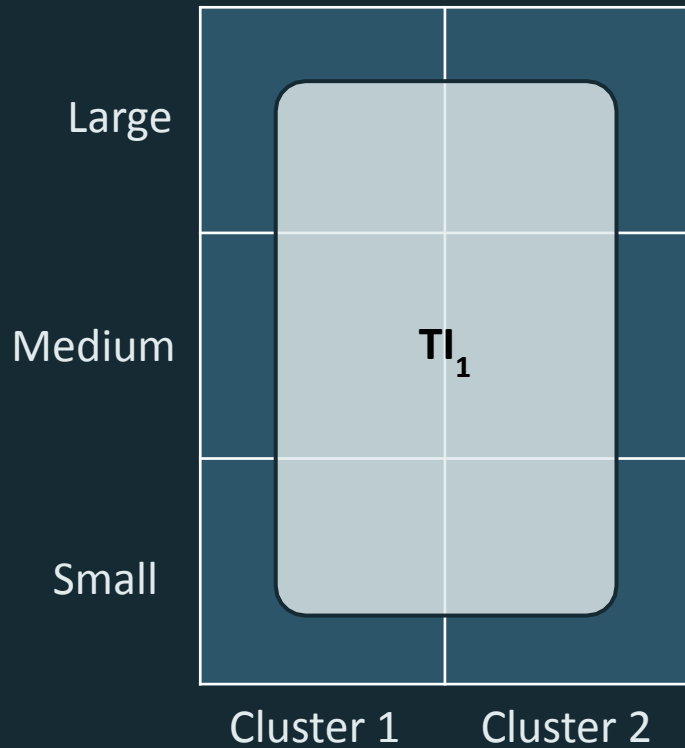
Scoping the tuning problem

- Workload characteristics (context)
 - Workload size = {Small, Medium, Large}
 - Cluster ID = {1, 2}



Scoping the tuning problem

- Workload characteristics (context)
 - Workload size = {Small, Medium, Large}
 - Cluster ID = {1, 2}

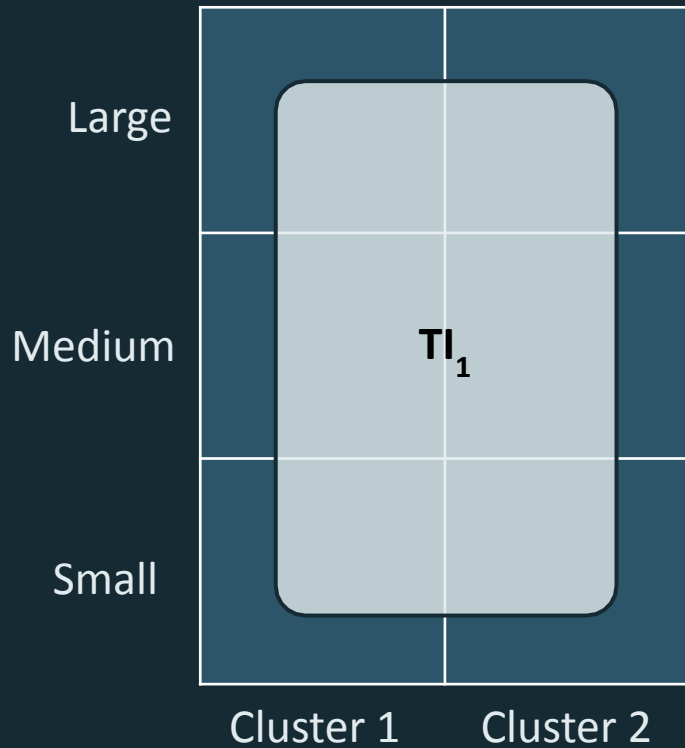


?

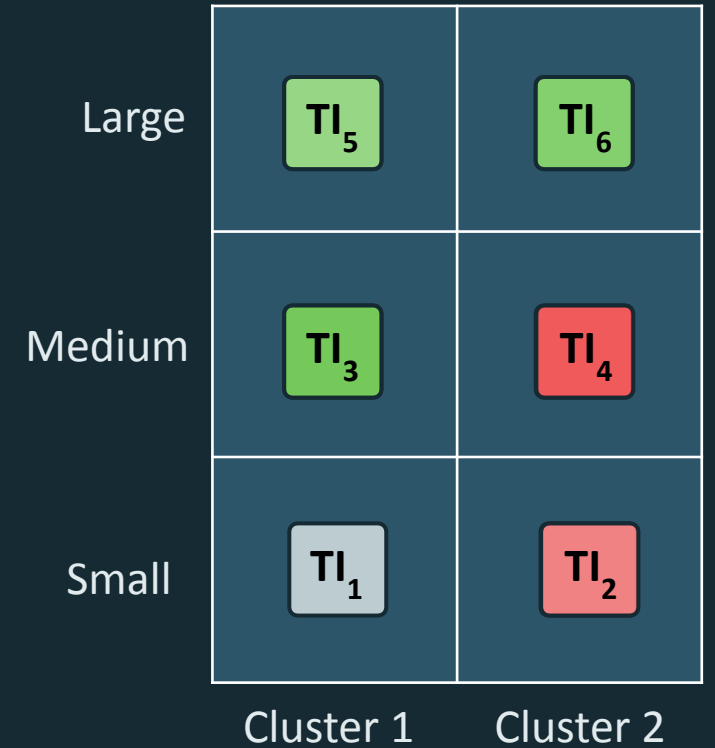


Scoping the tuning problem

- Workload characteristics (context)
 - Workload size = {Small, Medium, Large}
 - Cluster ID = {1, 2}

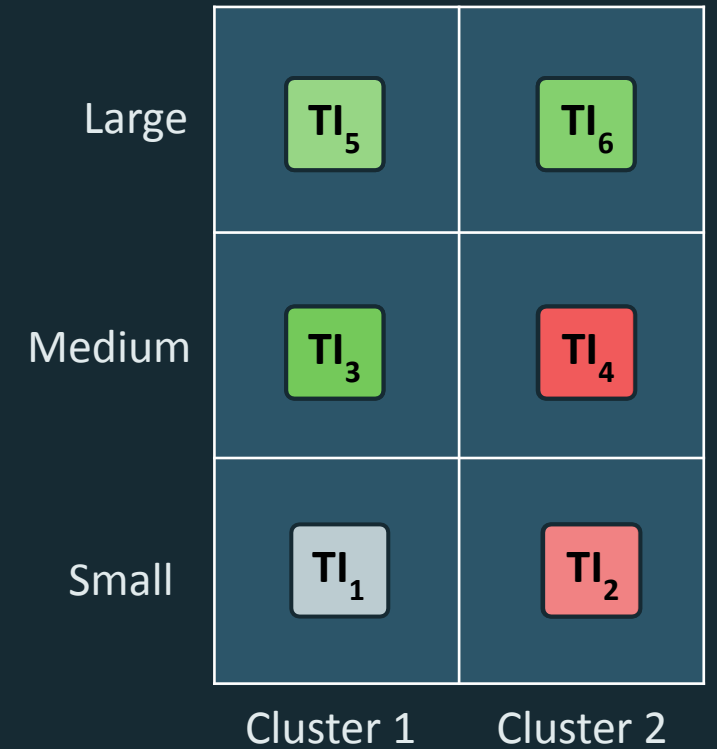
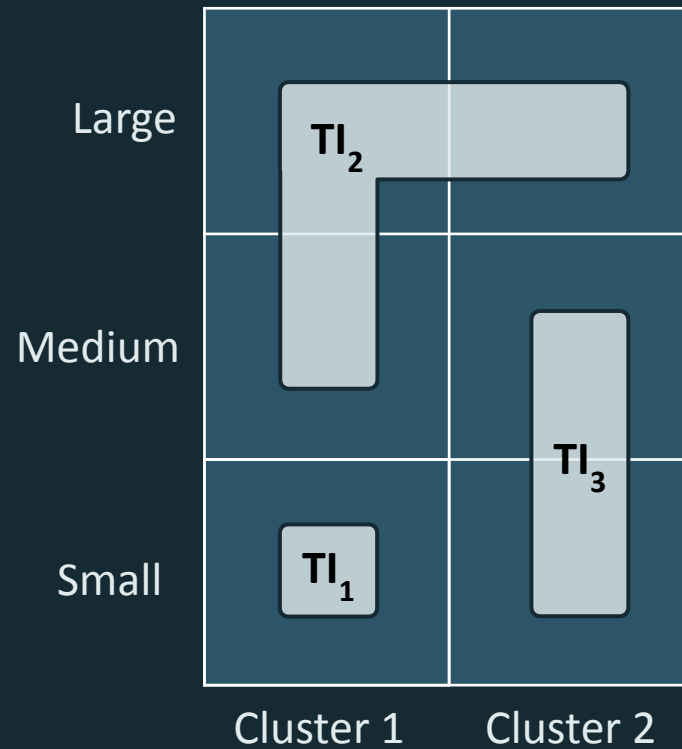
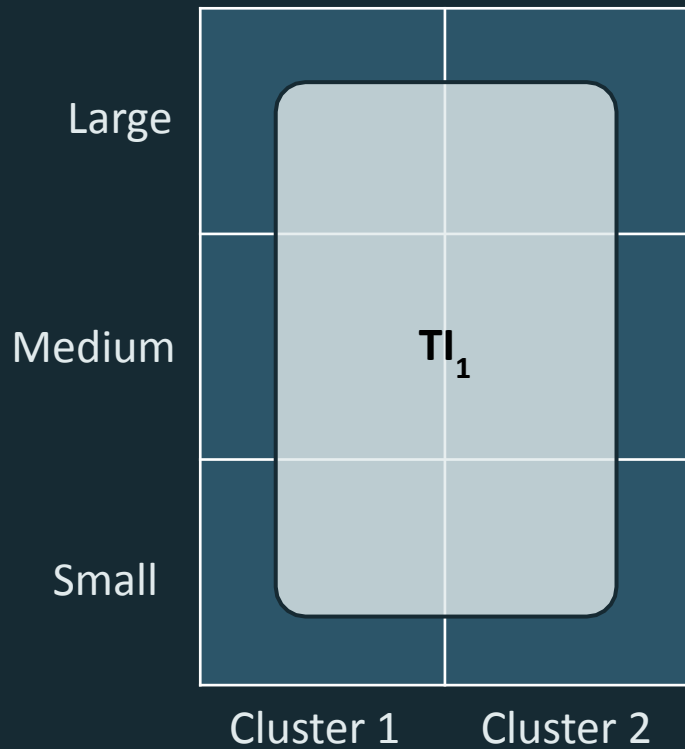


?

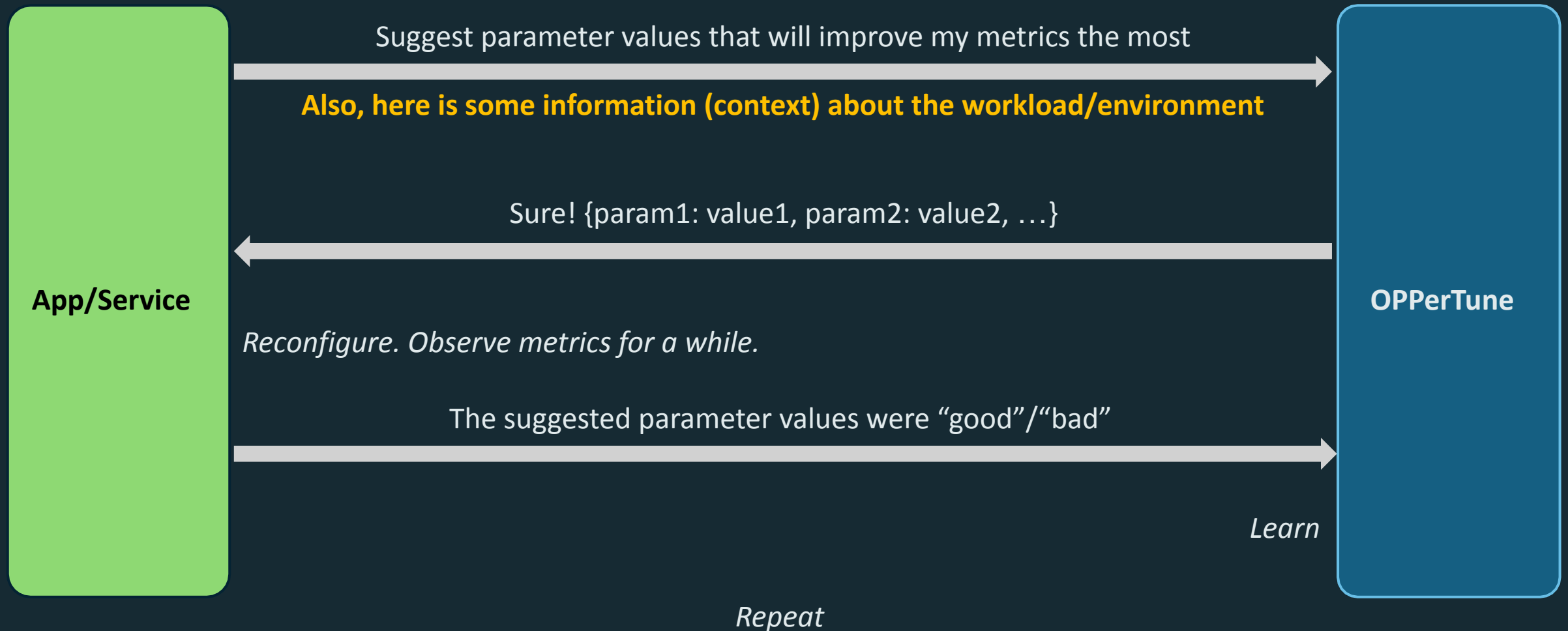


Scoping the tuning problem

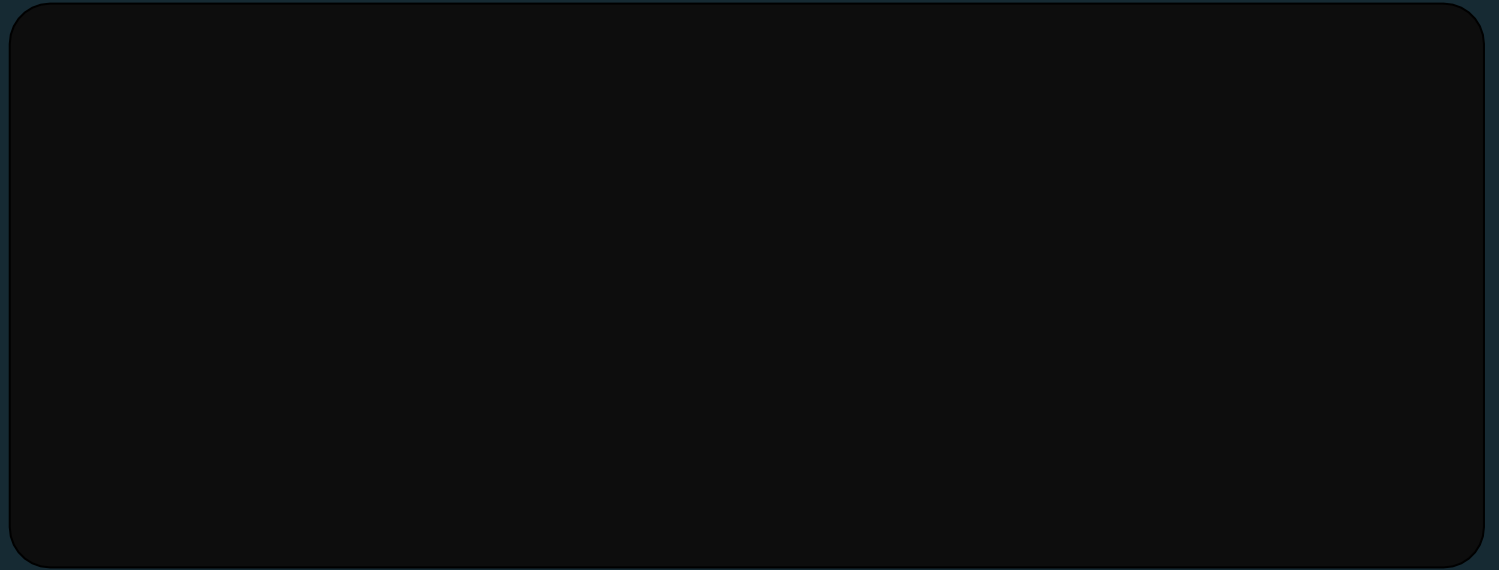
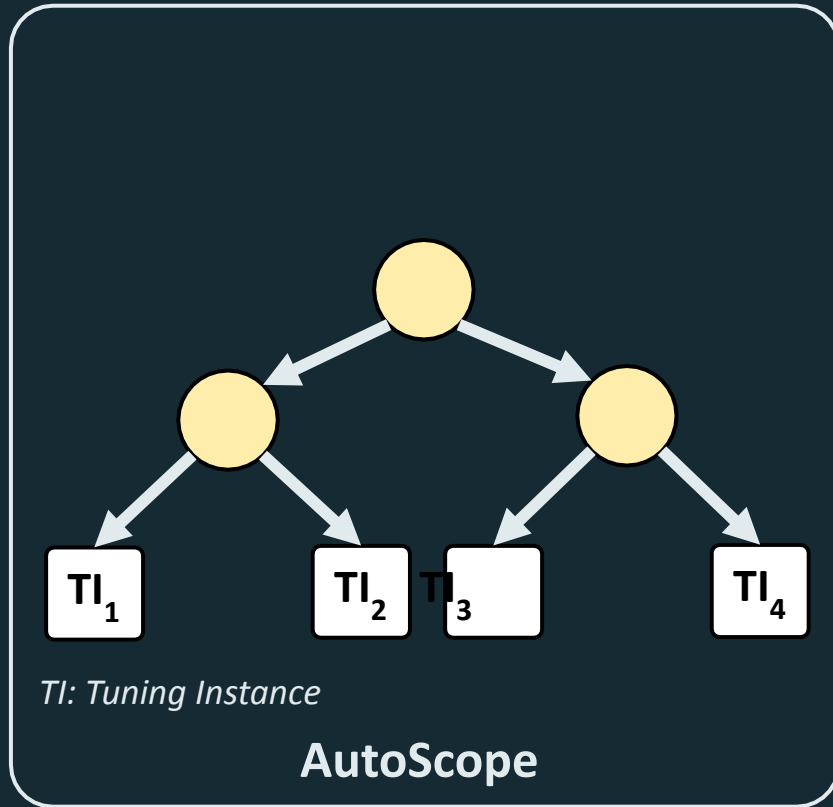
- Workload characteristics (context)
 - Workload size = {Small, Medium, Large}
 - Cluster ID = {1, 2}



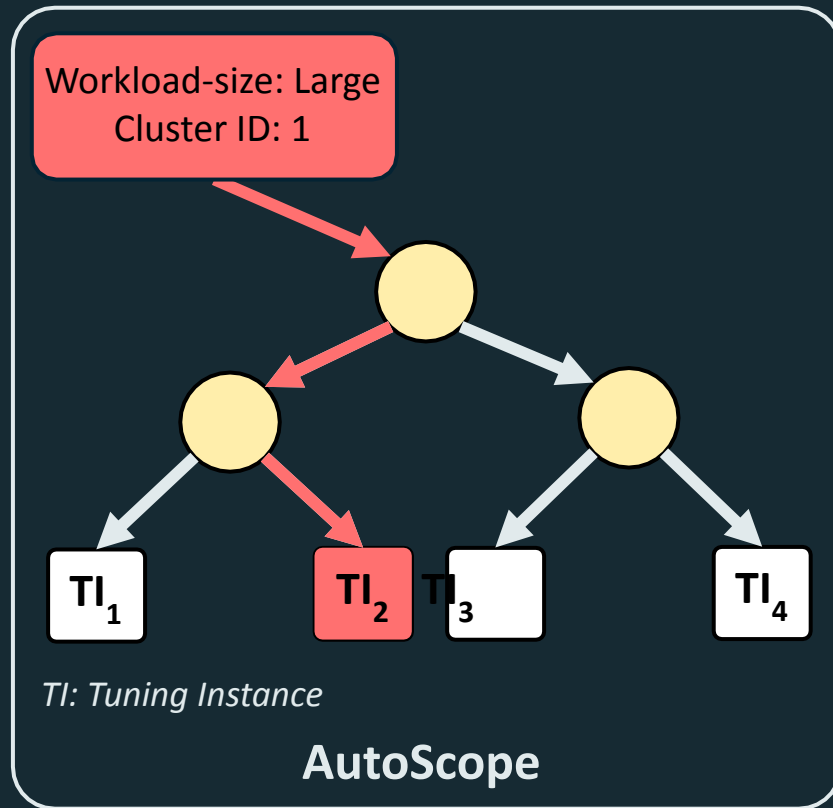
AutoScope



AutoScope

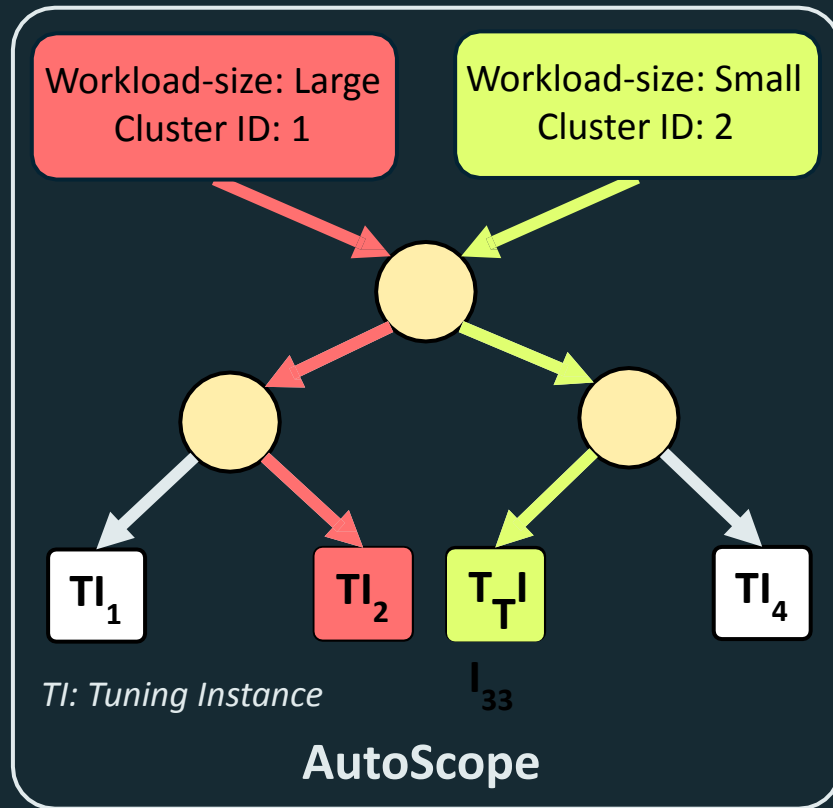


AutoScope



```
if workload_size == "large" and cluster_id == 1:  
    return tuning_instance_2.predict()  
    # scan_interval=40, expander="priority"
```

AutoScope



```
if workload_size == "large" and cluster_id == 1:  
    return tuning_instance_2.predict()  
    # scan_interval=40, expander="priority"  
  
elif workload_size == "small" and cluster_id == 2:  
    return tuning_instance_3.predict()  
    # scan_interval=30, expander="random"  
  
elif ...
```

Effectiveness of AutoScope

- Deployed SelfTune and OPPerTune for 2 weeks in 2 production clusters (workloads could be re-run on the clusters, so we could test both the frameworks in the timeframe)

• 1

Method	Experiment Completion Time (in minutes)		Sample Complexity (#rewards)	
	Cluster 1	Cluster 2	Cluster 1	Cluster 2
Pre-deployment choices	105.85 $\pm$ 16.75	36.66 $\pm$ 1.60	-	-
Expert choices	42.41 $\pm$ 5.28	34.46 $\pm$ 4.72	-	-
SelfTune _{cluster, type, size}	38.56 $\pm$ 6.55	30.79 $\pm$ 0.52	94	313
AutoScope	38.98 $\pm$ 5.90	32.71 $\pm$ 0.26	29	61

Takeaway

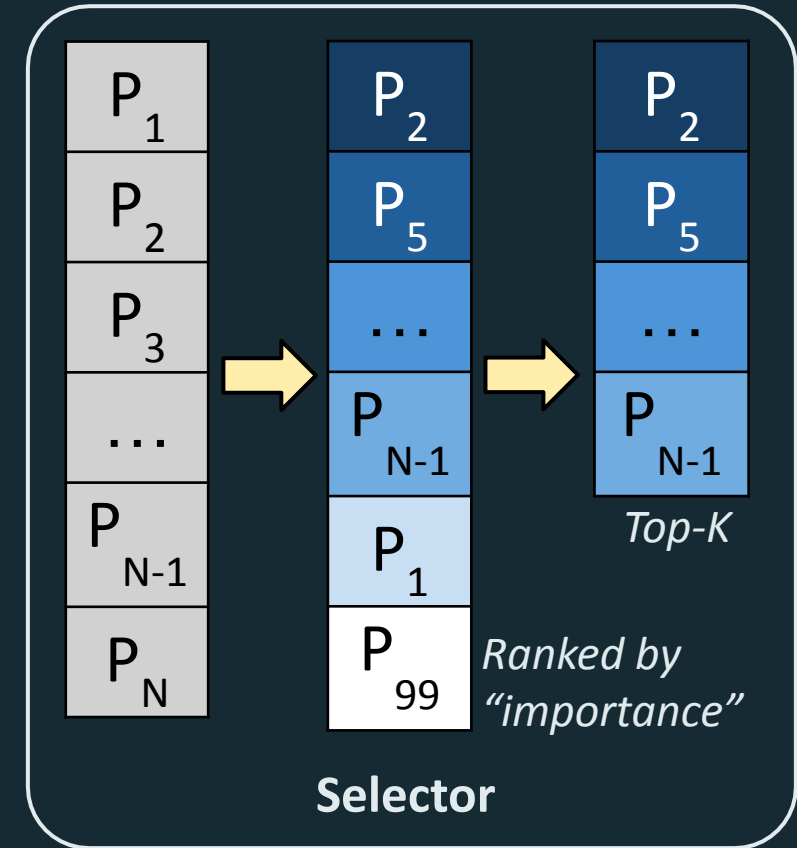
AutoScope reduces completion times significantly with less than a third of samples (i.e., in less than a third of time)

Challenge 3

Filtering parameters to tune

Microbenchmarking – Mitigating the cost of tuning

- Apps can have 1000s of parameters
- Tuning all of them will be slow
- Can also be costly
 - E.g., service disruptions

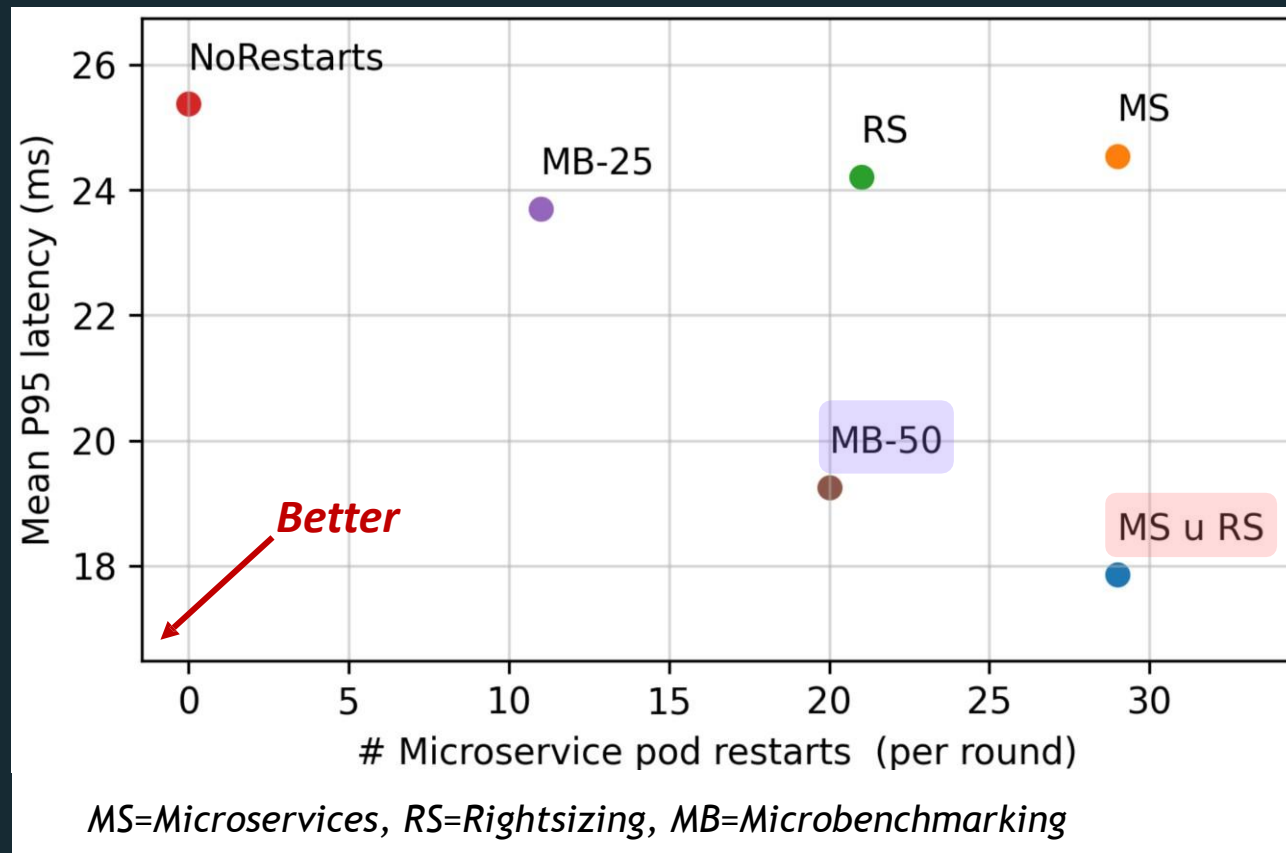


Effectiveness of Microbenchmarking

- **Workload:** Constant RPS (here, 2800) on the DeathStarBench Social Network app
- Fixed budget of 50 rounds
- 1 round means
 - Configuring the app with the parameters returned by the algorithm
 - Running workload for 20 minutes
 - Measuring the P95 latency

Takeaway

Microbenchmarking achieves a good tradeoff between performance and the cost of tuning



Summary

- Parameter tuning for deployed apps is hard
 - Large parameter space
 - Hybrid parameters across the software stack
 - Determining the right scope
 - Deciding which parameters to tune
- OPPerTune helps address all these
 - Propose novel algorithms to tackle these challenges
 - General end-to-end tuning service
 - Easy to setup and integrate, and is scalable
 - Is open-sourced (github.com/microsoft/oppertune)

Reflection

- Heavy reliance on default values:
 - Microbenchmarking is done wrt default values.
- All performance results are done after OPPerTune converges.
 - What happens in the time between deployment and convergence?
 - Is the new feature buggy or is OPPerTune causing problems?

Resources

- Project homepage
 - aka.ms/oppertune
- Repo
 - github.com/microsoft/oppertune
- Related papers
 1. [SelfTune: Tuning Cluster Managers](#) (*NSDI 2023*)
 2. [Learning Accurate Decision Trees with Bandit Feedback via Quantized Gradient Descent](#) (*TMLR 2022*)
 3. [Optimal regret algorithm for Pseudo-1d Bandit Convex Optimization](#) (*PMLR 2021*)



aka.ms/oppertune